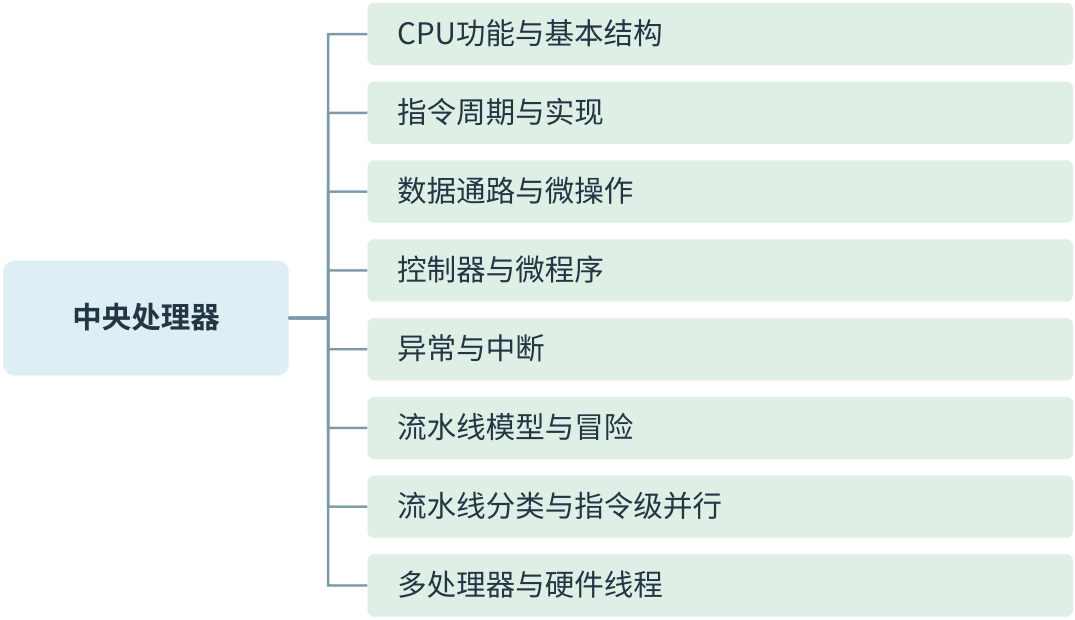
中央处理器 5

章节脉络



课程连接·先看一次指令

跟踪指令从取指、译码、执行到可能的访存与写回；每一步由控制信号选择相应数据通路。

5.1 从指令语义到执行组织

课件先用 CPU 功能与结构图识别寄存器、运算器和控制器，再用指令执行导图连接取指、间址、执行和中断阶段。本章沿这条路径先解释一条指令怎样得到正确结果，随后把流水线、多发射、多处理器与硬件线程接入同一章，始终保留“值在何时产生、资源在何时被占用”的约束。

定义 5.1.1 (CPU 的功能与实现层次)

CPU 承担指令控制、操作控制、时间控制、数据加工和规定的异常/中断处理。运算器处理数据，控制器解释并组织执行，寄存器保存跨时刻状态；数据通路承载数据传送与运算，控制部件决定何时、按何种方式使这些动作生效。

ALU、累加器 ACC、通用寄存器组、移位器和计数器等可出现在运算部件的教学模型中；PC、IR、指令译码器、时序系统和微操作信号发生器等常放在控制器模型中。实际 CPU 划分依实现；例如 PSW 既包含运算标志，也包含处理器状态，不能仅凭图上位置认定其全部位都允许用户程序任意写。

课件把寄存器分为程序员可见和实现内部两层：通用寄存器、PC 及规定的程序状态可进入体系结构描述；MAR、MDR、IR 和临时寄存器通常属于实现内部。这里“可见”指软件模型的一部分，是否可由某条用户指令直接读取或写入仍由 ISA 与权限规定。

5.2 寄存器的名字说明它暂存什么

最小反例·地址宽与数据宽可不同

若按字节编址、可寻址 64MiB，地址需要 26 位；若一次访存交换 32 位，MDR 可为 32 位。MAR 为 26 位而 MDR 为 32 位完全自洽，不能因二者都接主存就要求等宽。

CPU 的基本任务是取指、解释指令并控制操作执行，同时处理规定的异常和中断。运算部件完成算术逻辑处理，寄存器保存当前操作需要的状态，控制器安排数据通路中的动作。每个部件的作用要放到一次实际传送中理解。

寄存器	保存内容及用途
PC	与后继取指位置有关的地址；更新规则由执行时序决定
IR	当前正在译码或执行的指令编码
MAR	本次主存访问的地址
MDR	与主存交换的指令或数据
通用寄存器组	指令能按编号访问的操作数、地址或临时结果
PSW/标志寄存器	运算标志及体系结构规定的处理器状态

“寄存器在 CPU 内” 不等于 “它属于通用寄存器组”。PC、IR 与通用寄存器的用途不同；MAR、MDR 在题目的抽象模型中标明访存接口，但这并不意味着所有现代芯片都用同样的名字设置独立物理寄存器。

MAR 位数由可寻址单元数决定，MDR 位数由该数据通路一次访存交换的位宽决定。若数据总线 32 位、按字节编址且有 26 位地址，MDR 可以为 32 位而 MAR 为 26 位，两者无须相等。MDR 刚完成取指时存指令，刚完成数据读时存操作数，它的内容随本次访存的类型改变。

5.2.1 例 5-1　哪些元件负责运算，哪些元件保存状态

例 5.2.1 (配套辨析)

【配套例题】数据通路由组合逻辑元件（操作元件）和时序逻辑元件（状态元件）组成。以下给出的元件中，属于操作元件的是（　）。

I. 算术逻辑部件（ALU）　II. 程序计数器（PC）
III. 通用寄存器组（GPRs）　IV. 多路选择器（MUX）

A. 仅 I、II　B. 仅 I、IV　C. 仅 II、III　D. 仅 I、II、IV

解析。ALU 按当前输入产生运算输出，MUX 按选择信号把某路输入送往输出，二者是组合逻辑。PC 和通用寄存器组保存跨时钟的状态，属于状态元件，选 **B**。寄存器组内部也有组合读出或选择电路，但这不改变它作为状态元件的分类。

5.3 指令周期、机器周期与时钟周期

时钟周期是同步 CPU 中状态推进的基本时间单位。指令周期是从一条指令开始执行到完成所需的整个过程。机器周期是某些实现中为取指、访存等基本操作划分的较大时间单位，不同机器的划分不同，不应把它当成跨体系结构固定的常数。

指令周期可能含取指、间址、执行、中断等阶段；不是每条指令都经过所有阶段。寄存器指令无需为操作数访问主存，间接寻址可能增加取有效地址的阶段，外部中断也只有在满足响应条件时才进入响应过程。

单周期 CPU 把一条指令的全部操作安排在一个时钟周期内，所以时钟必须容纳最慢指令的数据通路；简单指令也使用同样长的周期。多周期 CPU 把指令拆成若干步，周期由各步关键路径决定，不同指令使用不同步数。流水线让多条指令的不同阶段重叠，减少连续指令的平均完成间隔，但未必缩短一条指令自身的延迟。

比较维度	单周期	多周期	简单标量流水线
同一时刻处理的指令	一条	一条的当前步骤	多条各在不同阶段
时钟受什么路径约束	最慢完整指令	最慢基本步骤	最慢流水段及段寄存器开销
一条指令占几拍	1 拍，但该拍较长	随所需步骤变化	通常跨多拍；稳态可每拍完成一条
同一运算资源能否复用	同拍需要的不同用途要避免争用	可跨步骤复用	重叠阶段同拍所需资源要避免争用

例如 PC 增量和数据加法在同一长周期内都需要求和时，不能仅因“单周期一拍”就假定一个无复用路径的 ALU 能同时完成两项操作；多周期可以在不同拍复用，流水线则要核查两者是否会同时发生。CPI=1 的单周期与稳态 CPI 约 1 的流水线可能有完全不同的时钟长度，比较性能仍须回到执行时间。

若题目给“每次访存 2 周期”，应把它安排到取指和操作数读写等实际位置；不能拿“某类指令固定 3 个机器周期”代替具体数据通路的资源分析，实际步数仍以资源限制为准。

不变量·总线同拍只有一个驱动源

单内部总线上，R1 送 Y 与 R2 送 MAR 要求两个不同值同时驱动，发生冲突；R1 同时送 Y 和 MAR 则可能允许，前提是两目标都具备写入路径和独立写使能。

5.4 一条指令怎样变为微操作

微操作是数据通路能完成的基本动作，例如 PC 送 MAR、两寄存器值相加、结果写回。控制信号选择输出源和 ALU 功能，允许某个寄存器写入，或发出存储器读写请求。微操作是动作，微命令是发出该动作的控制命令，两者不是同一个对象。

下例采用单条 CPU 内部总线。一次总线传送只允许一个驱动源，但同一值可以同时送入多个目的寄存器。ALU 的一输入来自暂存器 Y，另一输入来自总线，结果先存 Z，再由 Z 上总线写回。这里先按图确认资源，再写时序，不另补未画出的专用路径。

> **按图排微操作**
>
> 先把机器指令写成寄存器传送语义；再列出每一步的源、目的和可用路径；同拍检查总线驱动冲突、端口数量及写使能；最后加入访存等待与结果可用时刻。若题目没有画某条旁路，就不能为了缩短周期自行增设。

5.4.1 例 5-2　从单总线图确定控制信号与取指时序

【配套例题】某 CPU 中部分数据通路如下图所示，其中，GPRs 为通用寄存器组；FR 为标志寄存器，用于存放 ALU 产生的标志信息；带箭头虚线表示控制信号，如控制信号 Read、Write 分别表示主存读、主存写，MDRin 表示内部总线上数据写入 MDR，MDRout 表示 MDR 的内容送内部总线。

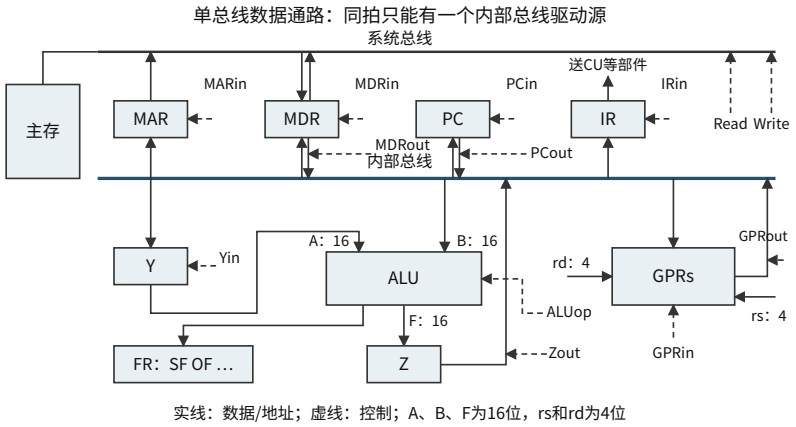


Figure 5.1: 单条内部总线连接 Y、ALU 与 Z；rs、rd 为 4 位寄存器编号

（1）设 ALU 的输入端 A、B 及输出端 F 的最高位分别为 A15、B15 及 F15，FR 中的符号标志和溢出标志分别为 SF 和 OF，则 SF 的逻辑表达式是什么？A 加 B、A 减 B 时 OF 的逻辑表达式分别是什么？要求逻辑表达式的输入变量为 A15、B15 及 F15。

（2）为什么要设置暂存器 Y 和 Z?

（3）若 GPRs 的输入端 rs、rd 分别为所读、写的通用寄存器的编号，则 GPRs 中最多有多少个通用寄存器？rs 和 rd 来自图中的哪个寄存器?已知 GPRs 内部有一个地址译码器和一个多路选择器，rd 应连接地址译码器还是多路选择器?

（4）取指令阶段（不考虑 PC 增量操作）的控制信号序列是什么?若从发出主存读命令到主存读出数据并传送到 MDR 共需 5 个时钟周期，则取指令阶段至少需要几个时钟周期?

（5）图中控制信号由什么部件产生？图中哪些寄存器的输出信号会连到该部件的输入端?

解析。(1) SF 直接记录结果最高位，所以 SF $= F_{15}$。OF 判断有符号运算是否越界。为简写，以下令 $a = A_{15}, b = B_{15}, f = F_{15}$，乘号表示逻辑与、加号表示逻辑或、上横线表示逻辑非：

$$\mathrm{OF}_{\mathrm{add}} = ab\overline{f} + \overline{a}\,\overline{b}f, \qquad \mathrm{OF}_{\mathrm{sub}} = a\overline{b}\,\overline{f} + \overline{a}bf.$$

加法只在两输入同号、结果异号时溢出；减法只在两输入异号、结果符号不同于 A 时溢出。例如 16 位 32767 加 1 得到 8000H，a=b=0、f=1，代入加法表达式得 OF=1；−32768 减 1 得到 7FFFH，a=1、b=0、f=0，代入减法表达式也得 1。SF 只保存结果的最高位，并不保证有符号运算的结果在数学上正确。

（2）同一时刻内部总线只能提供一个值，而 ALU 要同时接收两个输入。先把第一个输入存 Y，总线随后送第二个输入到 B 端，Y 提供 A 端。ALU 产生结果时总线还在送第二输入，若直接把结果也送回同一总线会争用；先用 Z 保存，下拍再由 Z 输出。Y 解决两个输入先后取得的问题，Z 解决结果输出的时机问题；二者并不是两个名字不同的通用寄存器。

（3）rs 和 rd 均为 4 位编号，最多 $2^4 = 16$ 个通用寄存器。它们来自指令寄存器 IR 中的对应字段。rd 决定本拍把结果写入哪一个寄存器，接写地址译码器；rs 决定从哪个寄存器读出，控制读出多路选择器。4 位编号能选 16 个寄存器，不表示每个寄存器只有 4 位。

（4）先用 PC 提供指令地址，访问主存，最后将读出的指令送 IR：

最小反例·同一动作不可重复计延迟

若题给从 Read 到主存数据送 MDR 共 5 拍，这 5 拍已经含送入 MDR。再加一次 “写 MDR” 会重复计数；但随后 MDR 经内部总线写 IR 仍是另一项真实传送。

控制步骤	有效控制信号	数据变化与所需时间
地址准备	PCout、MARin	PC 经内部总线写 MAR，1 周期
主存读	Read	主存按 MAR 地址读出，送 MDR，共 5 周期
装入指令	MDRout、IRin	MDR 经内部总线写 IR，1 周期

错项排除·编码对象先分清

32 条机器指令只需 5 位区分操作码，不说明控制存储器地址也只需 5 位。公共 2 步加每指令 4 步共有 130 位置，微地址需要 8 位；操作码、微地址和控制位数分别统计。

因此至少 $1+5+1=7$ 个时钟周期。5 周期已包含主存读出并送 MDR，无须再加一次 “写 MDR” 的额外周期。图中 MDRin 专指从内部总线写 MDR，并不出现在这条外部主存读入路径上。题目明确不考虑 PC 增量，也不必为更新 PC 多加一步。

（5）控制信号由控制器中的控制部件 CU 产生。IR 输出指令的操作信息，FR 输出 SF、OF 等状态，二者共同为控制决策提供输入。通用寄存器保存的是程序数据，PC 提供执行位置，不能因为它们在 CPU 内，就把所有寄存器都列为本图 CU 的判断输入。

5.4.2 例 5-3　间接操作数经过哪些部件

【配套例题】某指令功能为 $\mathrm{R}[r2] \leftarrow \mathrm{R}[r1] + \mathrm{M}[\mathrm{R}[r0]]$，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（　）。

I. 通用寄存器组（GPRs）　II. 算术逻辑单元（ALU）
III. 存储器（Memory）　IV. 指令译码器（ID）

A. 仅 I、II　B. 仅 I、II、III　C. 仅 II、III、IV　D. 仅 I、III、IV

解析。r1 的值和 r0 中的地址来自寄存器组；按 r0 值访问存储器得到第二个操作数；ALU 做加法并写回 r2，所以 I、II、III 必需，选 **B**。题目已限定 “取数及执行”，指令译码是此前确定操作的工作，不能把整个指令周期使用过的所有部件都加进来。

解数据通路题时，先写出指令的寄存器传送语义，再用图上的实际路径连接源和目的，最后安排周期。总线冲突、写入条件和内存延迟通常比 “背一张步骤表” 更能决定答案。不要为了省周期自行补出未画出的旁路。

5.5 控制信号由组合逻辑还是微程序产生

硬布线控制器根据操作码、当前节拍和状态条件，经译码与逻辑电路产生控制信号。它不必逐条读取控制存储器中的微指令，常有较短控制延迟；新增复杂指令可能需要修改更多逻辑。

微程序控制器把一条机器指令的控制过程组织为微指令序列，保存在控制存储器中。微指令的控制字段产生本拍的微命令，后继地址及条件字段决定下一条微指令。控制存储器通常是 CPU 内部的专用存储器，不是用户主存；微程序计数器或微地址寄存器也不是机器程序的 PC。

两个控制器实现同一 ISA 时，对同一机器指令应产生等价的软件可见结果；差别在于控制序列由逻辑电路直接形成，还是从控制存储器取出微指令形成。机器指令的操作码用于选择一种指令功能，微地址用于选择控制序列中的一步，微命令控制具体部件动作。因而 “机器指令有 32 条”“控制存储器有 130 个位置”“本拍需发 5 条微命令” 数的是三个不同对象，各自的位数不能互相套用。

微指令的直接控制方式为每条控制线分配一位，便于同拍并行，但字宽较大。字段直接编码把互斥微命令分组，每字段经译码选一个命

令，需要保留 “不发该组命令” 的编码。字段间接编码还借助其他字段共同解释。水平型倾向于显式并行多个微命令，垂直型以较短编码表示操作，往往需要更多微指令才能完成同一控制任务；不能笼统地说后者并行性更高。

最小反例·4 条命令不只 4 种状态

一个互斥字段含 4 个微命令，仍需 “不发该组命令” 的第 5 种状态，所以通常至少 3 位。互斥只限制同字段同时发一条，不能省掉本拍暂不用该字段的状态。

5.5.1 微程序控制器的状态怎样推进

控制存储器 CM 保存微指令；微地址寄存器 CMAR（也常称微程序计数器）保存下一次要访问的微地址；读出的微指令进入 CMDR/微指令寄存器，其中控制字段经直接控制或译码发出微命令。后继逻辑依据下地址字段、条件和节拍形成下一微地址。CMAR 与程序 PC 分别指向控制存储器和机器程序空间，不能共用 “指令地址” 的名称而忽略所属层次。

后继微地址可由断定法读取下地址字段，也可顺序递增；机器指令操作码用于确定执行微程序入口，取指或中断等公共微程序入口可由硬件规则给出。课件用固定 0 号作为公共取指入口，是该教学模型的约定，不是所有微程序 CPU 的统一物理地址。

水平、垂直与混合微指令是控制字组织的分类。直接控制用较多位明确允许并行微命令；字段编码用较短编码选择互斥动作；混合形式在紧凑编码与并行控制之间折中。“字短” 不等于 “一条机器指令执行更快”，应比较微周期数与各微周期时长。

5.5.2 例 5-4　互斥类为什么还要一个空操作编码

【配套例题】某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控制字段至少有（　）。

A. 5 位　B. 6 位　C. 15 位　D. 33 位

解析。每组除了要能发出其中一条命令，还要能一条都不发。各组位数为 $\lceil \log_2(7+1) \rceil = 3$、2、4、3、3，总计 15 位，选 **C**。例如若某组有 4 条命令，2 位只有 4 种编码，没有 “本拍不用该组” 的位置，通常要 3 位。

5.5.3 例 5-5　控制存储器地址字段怎样估位宽

【配套例题】某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是（　）。

A. 5　B. 6　C. 8　D. 9

解析。公共取指微程序只存一份，控制存储器至少需 $32\times4+2=130$ 个微指令位置，地址至少 $\lceil \log_2 130 \rceil = 8$ 位，选 **C**。32 种机器指令只需要 5 位编码，不表示微指令的存储地址也只需 5 位；两者编码的对象不同。

取指虽然不靠 “上一条指令的操作码” 决定读哪条指令，但仍需要控制信号来选择 PC、发出存储器读并写入 IR。流水线图有时省略这些固定发生的控制动作，但这不代表取指、译码电路不需要任何控制。题目说 “数据通路不包含控制部件” 时，是按功能划分数据处理通路与信号生成部分；这也不意味着数据通路能脱离控制运行。

5.6 异常、中断与返回位置

课程连接·缺页使当前指令重试

OS 调入缺页后，原访问尚未完成，处理器通常从该指令重试；设备完成中断对应一条已完成指令的边界。返回点取决于事件与 ISA 的精确状态规则，不由当前取指 PC 猜测。

本讲义按 408 的常用语境区分：异常与当前执行的指令有关，在执行过程中检测，例如缺页、非法指令、算术异常；外部中断来自指令执行之外的事件，例如设备完成、定时器请求，通常在指令边界接受响应。广义的 “中断机制” 有时统称两者，遇到题目要先辨清其用词范围。

故障 fault 通常可在修复后重新执行，例如缺页后调入页面；陷阱 trap 是有意发出的通知或调试事件，处理完后通常从后继位置继续；终止 abort 表示严重情况，通常不能可靠恢复。不同 ISA 的异常分类和保存 PC 的细则不同，此处不把每一种算术异常固定归入同一类别。

响应异常或中断时，硬件需保存足够的返回信息、切换到规定状态并找到处理程序入口。通用寄存器现场往往由入口汇编代码和软件继续保存。中断服务例程是软件；“中断隐指令” 则是对硬件自动响应动作的教材称呼，不是程序员在代码中插入的一条普通指令。

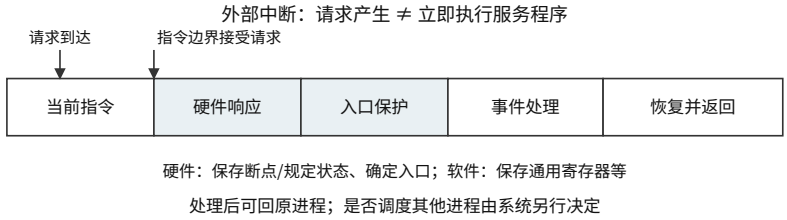


Figure 5.2: 外部请求先等待响应边界，再由硬件入口转入软件处理

5.6.1 例 5-6　缺页后为什么重试当前指令

【配套例题】异常事件在当前指令执行过程中进行检测，中断请求则在当前指令执行后进行检测。下列事件中，相应处理程序执行后，必须回到当前指令重新执行的是（　）。

A. 系统调用　B. 页缺失　C. DMA 传送结束　D. 打印机缺纸

解析。页缺失意味着当前指令需要访问的页面尚未就绪，指令的原定访问还未完成。处理程序调入页面后，应重新执行这条指令，选 **B**。系统调用已经发出了服务请求，不能不加区分地重做调用指令；DMA 完成和打印机事件来自外设，不要求重做某条已完成的普通指令。

外部中断的返回地址通常是下一条尚未执行指令的位置，不能写成 “刚执行完的那一条”。发生故障时可能保存当前指令起点，发生陷阱时可能保存后继位置。给出流水线或变长指令的题目，还要看体系结构怎样保证可恢复状态，不能单凭此时取指 PC 的数值推断精确断点。

处理器进入内核态不必切换到另一个进程。一次设备中断可能处理后返回原进程；调度器也可能决定切换执行主体。模式切换与进程上下文切换相关但不等价。第 7 章进一步讨论设备请求、屏蔽和中断嵌套。

5.6.2 基本控制巩固

1. 【配套例题】相对于微程序控制器，硬布线控制器的特点是（　）。A. 指令执行速度慢，指令功能的修改和扩展容易；B. 指令执行速度慢，指令功能的修改和扩展难；C. 指令执行速度快，指令功能的修改和扩展容易；D. 指令执行速度快，指令功能的修改和扩展难。
2. 原创补充:沿用本章单总线图,能否在一拍同时执行 `R1→Y`、`R2→MAR`? 能否同时执行 `R1→Y`、`R1→MAR`?

答案。1. D，比较的是教材所述常见控制实现的控制开销与修改难度。2. 前者不能，两源争用总线；后者可以，同一来源同时送两个写入端，只要各目的具备独立写使能。

不变量·第一条付满延迟，后续按间隔

理想 k 段、n 条指令总时长为 (k+n−1) 拍：第一条需 k 拍，剩余 n−1 条每拍完成一条。不能给每条都加一次填充延迟，也不能把阶段数直接当任何基线下的加速比。

5.7 延迟和完成间隔是两个量

把一条指令分为 k 段，每段使用自己的部件或占用自己的时段，让多条指令的不同阶段重叠执行，称为指令流水线。同一条指令内部的先后依赖依然存在：必须先取得地址，才能读取相应数据。它主要提高连续指令的吞吐率。

定理 5.7.1 (理想标量流水线的总时间)

理想 k 段流水线，每段耗时一个时钟周期 τ，无停顿且每周期启动一条指令，完成 n 条指令需

$$T_{\text{pipe}} = (k + n - 1)\tau.$$

由第一条延迟与后续完成间隔计数

前 k 个周期使第一条指令通过全线，此后每周期再完成一条。填充时间只在开始付一次，并非每条指令各付一次。吞吐率为 n/T_{pipe}，n 足够大时趋近 $1/\tau$；单条延迟约为 $k\tau$。加入停顿 s 周期后，总时间变为 $(k+n-1+s)\tau$；这里假定每个停顿周期都使最终完成时间推后一个周期。

□

若非流水方案中每条指令依次完整走完这 k 段、每段也占用同样的 τ，则非流水时间为 $nk\tau$，加速比为 $nk/(k+n-1)$。若原方案时钟不同、各段耗时不等或另有资源复用，基线要重新计算；流水级数本身并不等于加速比。

5.7.1 例 5-7　最慢的一段决定时钟下限

【配套例题】某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90 ns、80 ns、70 ns 和 60 ns，则该计算机的 CPU 时钟周期至少是（　）。

A. 90 ns　　B. 80 ns　　C. 70 ns　　D. 60 ns

解析。相邻阶段寄存器在同一节拍推进，必须留足最慢段 90 ns，选 **A**。若实际阶段寄存器开销为 t_r，则时钟周期通常至少为 $\max_i(t_i) + t_r$；题目已说明忽略这项开销。

在本题时间下，理想完成 10 条指令需 $(4 + 10 - 1) \times 90 = 1170$ ns。若非流水方案每条直接耗时 $90 + 80 + 70 + 60 = 300$ ns，10 条为 3000 ns，加速比约 2.56；若基线规定每段都占 90 ns，基线时间又会不同。比较时必须明确分母采用哪种机器。

若每条指令都有效使用 k 个等时段，n 条共占用 nk 个“段 × 周期”，时空图总面积为 $k(k+n-1)$ 个“段 × 周期”，所以理想流水线效率为 $E = n/(k+n-1)$。有空闲段、停顿或不同功能指令时，应按实际有用面积重新计算，不能把这个理想式用于所有时空图。加速比比较两种方案时间，效率比较段的有效利用，二者不是同一个比例。

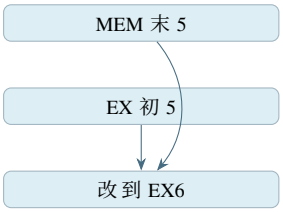
5.7.2 课件中的流水线分类

按使用层次，部件级流水线把算术运算拆段，处理机级流水线把指令过程拆段，处理机间流水线让多台处理机分担任务阶段。按功能可分单功能与多功能；按功能配置能否在同一时刻并行用于不同功能，分静态与动态流水线；按各段是否存在反馈回路，分线性与非线性。非线性流水线可能让一项操作重复经过某些段。

“动态流水线” 在上述分类中讨论功能配置，“动态调度” 则讨论就绪指令能否提前执行，不能仅因都含 “动态” 就当作同一技术。超标量、超流水与超长指令字又是多发射或细分阶段等技术维度，随后分别说明。

最小反例·尚未产生的值不能转发

LOAD 的值到第 5 拍 MEM 末才形成，下一条本应第 5 拍 EX 开头使用它，两者时刻不合。已有 MEM→EX 旁路仍需把消费者推至第 6 拍；“有转发” 不能抹去因果顺序。



同拍初早于同拍末，故需推迟消费者

5.8 阶段寄存器保存的不只是一份数据

五段教学流水线常写为 IF 取指、ID 译码/寄存器读、EX 执行或地址计算、MEM 访存、WB 写回。算术指令也可能经过 MEM 这一拍但不访问数据存储器；存储指令通常不在 WB 写通用寄存器。阶段名只描述一种通路划分，并不意味着每条指令都在各段做同一件事。

阶段之间的寄存器保存下一段所需的操作数、立即数、目的寄存器编号、指令地址及控制信息。若只传数据不传目的编号，结果就可能写到错误寄存器；若指令已经阻塞，写使能却继续向下游传递，就可能重复写入或错误访存。

时钟负责推进状态，控制信号决定状态能否写入、组合部件执行哪种功能。阶段寄存器常需写使能、保持和清空控制，因此 “控制仅作用于组合部件、时钟仅作用于寄存器” 的割裂描述不能当作普遍规则。气泡是在某阶段插入一个不改变有效状态的操作，冻结则让原状态保持，两者效果不同。

5.9 冒险先按冲突对象分类

结构冒险来自同一拍内争用数量不足的硬件资源。例如取指和数据读取同时需要单端口统一存储器；可通过停顿、增加端口或分离指令与数据存储来处理。寄存器组同拍读写是否冲突，取决于端口数及前后半拍的时序约定。

数据冒险来自指令间数据依赖与执行时序的矛盾。写后读 RAW：后指令读前指令产生的新值；读后写 WAR：后指令过早写破坏前指令尚未读取的旧值；写后写 WAW：两次写的完成顺序颠倒，使最终值错误。在固定五段、顺序发射、ID 读寄存器且 WB 顺序写回的简单流水线中，通常需要处理 RAW，而既定次序使 WAR、WAW 不会发生。变为乱序执行或不同延迟写回后，这个结论不再普遍成立。

控制冒险来自后继指令地址未确定，常见于条件分支、跳转、异常和中断处理。无条件转移仍需在某个时刻获得目标地址，因而也可能造成前端控制冒险。“不判断条件” 并不等于 “不会停顿”。

5.9.1 转发能省去等待写回，但不能送来尚未产生的数据

转发把已经形成的结果从阶段寄存器等位置直接送到后续指令使用点，不必等写回寄存器组。例如 ALU 结果在 EX 结束时产生，后继指令下一拍进入 EX，可以从 EX/MEM 寄存器取得该结果。

load 的数据通常到 MEM 结束才产生。紧随其后的指令若下一拍 EX 就需要该值，即使有转发也来不及，必须插入一个等待周期。转发路径是否通往存储数据端、分支比较端，还要看具体电路；不能一见“支持转发”就推断所有相关都已解决。

错项排除·段寄存器还要携带控制

只传操作数不传目的编号，结果可能写错寄存器；保持数据却让写使能继续推进，可能重复写入。流水段状态要同时说明值、目的、有效位与控制信息，不只数“几个数据寄存器”。

5.9.2 同一组 RAW，在不同实现中等到哪一拍

题设实现	消费者取得值的条件	排时序时使用的约束
无转发，寄存器同拍不能读写	必须等生产者 WB 结束，再在 ID 读	消费者 ID 至少比生产者 WB 晚 1 拍
无转发，寄存器同拍先写后读	生产者 WB 前半拍写，消费者 ID 后半拍读	可以让消费者 ID 与生产者 WB 同拍
ALU 结果向 EX 转发	生产者 EX 末已有结果，下一拍送消费者 EX	不必为该值等到寄存器 WB
load 结果向 EX 转发	生产者 MEM 末才有读出数据	消费者 EX 须在该 MEM 之后，紧邻 load 仍可能停 1 拍

“相关”由程序中谁产生、谁使用同一个值决定；“冒险是否导致停顿”还由结果产生点、使用点、转发路径和寄存器时序共同决定。因此，无转发题不能套用 load-use 只停一拍的结论，有转发题也不必机械等到所有源写回。STORE 的数据可能在 ID 读取、在 MEM 使用，分支可能在 ID 或 EX 比较，须先看清具体通路的约定。

5.9.3 例 5-8　装入后立即使用需要一个气泡

【配套例题】下列指令序列中，指令 I1 和 I3、I2 和 I3 之间发生数据相关。假定采用“取指、译码/取数、执行、访存、写回”五段流水线方式，那么在采用转发技术时，需要在指令 I3 之前加入（　）条空操作指令才能使这段程序不发生数据冒险。

```
I1: add  r1, r0, 1    // R[r1] <- R[r0] + 1
I2: load r3, 12(r2)   // R[r3] <- M[R[r2] + 12]
I3: add  r5, r3, r1   // R[r5] <- R[r3] + R[r1]
```

A. 3　B. 2　C. 0　D. 1

解析。I1 在第 3 拍 EX 末形成 r1 结果，到 I3 需要时已经可用；I2 到第 5 拍 MEM 末才有 r3 数据，而无停顿的 I3 本应在第 5 拍 EX 使用它，时间相撞。把 I3 的 EX 推迟到第 6 拍即可，因此选 **D**。

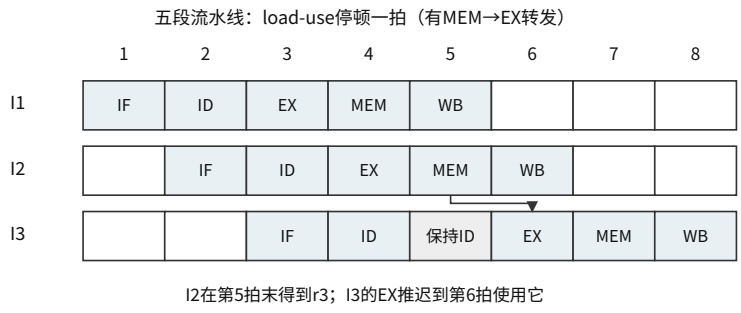


Figure 5.3: load 到 MEM 末才有数据；空泡把使用时刻推迟到下一拍

指令	成功推进的阶段与周期	等待状态
I1	IF1，ID2，EX3，MEM4，WB5	无
I2	IF2，ID3，EX4，MEM5，WB6	无
I3	IF3，ID4，EX6，MEM7，WB8	第 5 拍保持 ID，EX 为气泡

不变量·产生点必须早于使用点

先把每个值的产生段末与消费者所需段初标在时间线上，再检查是否有实际转发路径。STORE 数据端、分支比较端和 ALU 输入端可能不同，不能把一种 load-use 模板用于全部指令。

采用寄存器组同拍先写后读的时序，I1 第 5 拍 WB 写回的 r1 可在该拍被保持在 ID 的 I3 读到；若实现不同，须另有 WB 到 ID 的旁路。第 5 拍 EX 放入气泡，I3 在 ID 保持，下一拍取得装入结果。这里按题目所指的常见五段转发模型，把 r3 的使用点设在 EX；若某图要求在 ID 比较，所需等待和转发路径还可能不同。

5.9.4 例 5-9　同一段序列里同时有数据与控制等待

【配套例题】在采用 “取指、译码/取数、执行、访存、写回”5 段流水线的 RISC 处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1: add  s2, s1, s0  // R[s2] <- R[s1] + R[s0]
I2: load s3, 0(s2)   // R[s3] <- M[R[s2] + 0]
I3: beq  t2, s3, L1  // if R[t2] = R[s3] jump to L1
I4: addi t2, t2, 20  // R[t2] <- R[t2] + 20
I5: L1: ...
```

若采用转发（旁路）技术处理数据冒险，采用硬件阻塞方式处理控制冒险，则在指令 I1～I4 的执行过程中，发生流水线阻塞的指令有（　）。

A. 仅 I3　　B. 仅 I2、I4　　C. 仅 I3、I4　　D. I2、I3、I4

解析。I1 的 ALU 结果可以及时转发给 I2 计算地址，所以 I2 不因这项 RAW 等待。I2 是 load，I3 马上使用 s3，仍有 load-use 等待。I3 的分支结果未确定时，硬件阻塞后继 I4，形成控制等待。因此发生阻塞的是 I3、I4，选 **C**。分析一条序列不能只找到第一组相关就停下；每个使用值的位置和后继 PC 都要检查。

5.9.5 三种处理办法各改变什么

硬件阻塞保持相关指令及其前方的必要状态，在前级暂停、后级放气泡，让生产者先前进。编译调度把不依赖该结果、且移动后不改变程序语义的指令填进等待间隔；无法填满才用 NOP。不能把有副作用或可能越界访存的指令随意移过分支。

分支预测先选择一条路径继续取指，正确时减少等待，错误时清除错误路径上尚不能提交的结果并从正确位置重新取指。若每条指令发生预测错误的概率为 p、每次平均损失 d 周期，且其他开销已包含在基本 CPI 中，可用 $\mathrm{CPI} = \mathrm{CPI}_0 + pd$；“分支比例” 并不等于 “所有指令的预测失败比例”。

旁路主要让结果更早可用，不直接决定分支目标地址；某些设计把数据转发给分支比较器，缩短因数据相关而产生的分支等待，但不等于所有控制冒险都由旁路消除。题目需按具体控制路径判断。

5.10 多发射、动态调度与按序提交

最小反例·改名字不能提前算出值

前指令产生 r1，后指令确实需要这个结果，即使把目的映射为物理寄存器 p7，后指令仍须等 p7 得到值。重命名消除 WAR/WAW 名字冲突，不能消除真实 RAW 因果依赖。

超标量 CPU 每周期可发射多条指令，需要相应的译码、调度、执行和写回资源。是否真能每周期发满，取决于数据依赖、分支、功能部件和访存带宽。最大发射宽度 w 只给出理想吞吐率上限，不能把 w 直接当作任意程序的加速比。

超流水把阶段划得更细，试图缩短时钟周期。阶段寄存器开销和冒险损失会限制收益。超长指令字 VLIW 把若干可并行的操作交给编译器打包，硬件和指令格式仍需支持相应并行执行，并非 “只改编译器，硬件没有要求”。

动态调度可让后面的就绪指令先执行，减少因前面一条长延迟指令而造成的闲置。寄存器重命名为每次逻辑写入分配不同的物理寄存器，可去除 WAR、WAW 这类假相关，但不能消除真实 RAW：消费者必须等对应生产者的值。按序提交使软件看到的状态仍符合程序顺序，并帮助实现精确异常。

例如前指令还要读取旧 r1，后指令将写新 r1 时，为新值分配另一物理寄存器，可避免覆盖旧值；若后指令确实需要前指令算出的 r1，改名并不能凭空产生结果。区分 “名字相同” 和 “值确有依赖” 比背三组字母更有用。

5.11 一条指令处理多个数据，还是多个指令流

Flynn 分类按指令流和数据流区分：SISD 为单指令流单数据流；SIMD 为单指令流多数据流；MISD 为多指令流单数据流；MIMD 为多指令流多数据流。分类描述执行组织，不是按机器装了几个屏幕、几块内存来判断。

向量处理器用一条向量指令对一组数据元素重复执行同类操作，可采用向量流水和多个通道，属于常见 SIMD 组织。它有启动代价，长向量能更好地摊薄这笔代价；向量长度不等于同时存在同样多套完整处理器。SIMD 也不能自然地处理这样的任务：每个元素都要求完全不同的控制流程；分支发散可能降低有效利用率。

MIMD 允许不同处理单元执行不同指令流。共享物理地址空间的多处理器属于 MIMD 的一种，分布式存储系统也可属于 MIMD。SMP 中的 “对称” 是各处理器在系统功能等方面地位对等，不表示每个时刻执行同一条指令。

5.12 硬件线程、多核与共享内存

硬件多线程让一个核保存多套线程体系结构状态，以便在一个线程等待时运行另一线程。细粒度多线程可按周期交替线程；粗粒度多线程常在较长延迟事件时切换；同时多线程 SMT 允许同周期从不同线程发射操作，利用已有执行资源。它们都不要求芯片必须有多个核。

多核是在一个处理器芯片内集成多个执行核心。各核可以独立推进指令流，通常还共享某些 Cache、互连或内存控制资源。增加核数并不保证单线程加速，也不意味着功耗必然高于任意单核设计；工艺、频率、负载和资源共享均影响结果。

共享内存多处理器让各处理器使用同一物理地址空间，但各核寄存器、部分 Cache 仍可私有。共享地址空间只保证各处理器能访问同一对象，不保证多个读改写操作自动具有原子性。并发程序还需要同步，

Cache 一致性与存储顺序也要分别满足相应规则。操作系统的进程同步讲义会沿用这一边界。

课程连接·共享地址并不自动互斥

多核共享物理地址空间，仍可能并发读到同一旧值并分别写回。OS 同步中的原子操作与锁解决复合更新的协调；Cache 一致性不等于所有读改写序列自动原子。

5.12.1 例 5-10　硬件多线程不等于多核

【配套例题】下列关于并行处理技术的叙述中，不正确的是（　）。

A. 多核处理器属于 MIMD 结构
B. 向量处理器属于 SIMD 结构
C. 硬件多线程技术只可用于多核处理器
D. SMP 中所有处理器共享单一物理地址空间

解析。单核也能保存多套线程状态并共享执行部件，所以 C 错，选 **C**。A、B 是按指令流和数据流分类，D 说的是共享物理地址空间；都没有断言所有资源完全相同或所有访问自动同步。

5.12.2 例 5-11　无转发流水线的阻塞传播与最短调度

【配套例题】某 16 位计算机中，带符号整数用补码表示，数据 Cache 和指令 Cache 分离。下面给出了指令系统中部分指令的汇编格式及对应功能，其中 Rs 和 Rd 表示寄存器，mem 表示存储单元地址，(x) 表示寄存器 x 或存储单元 x 的内容。

```
ADD   Rs, Rd
SHL   Rd
SHR   Rd
LOAD  Rd, mem
STORE Rs, mem
```

上面每行给出一种汇编格式；下表逐项给出各助记符对应的名称和功能，不改变源、目的操作数的次序。

名称	助记符	指令功能
加法	ADD	(Rs) + (Rd) → Rd
算术/逻辑左移	SHL	2(Rd) → Rd
算术右移	SHR	(Rd)/2 → Rd
取数	LOAD	(mem) → Rd
存数	STORE	(Rs) → mem

该计算机采用 5 段流水方式执行指令，各流水段分别是取指 IF、译码/读寄存器 ID、执行/计算有效地址 EX、访问存储器 M 和结果写回寄存器 WB。流水线采用 “按序发射、按序完成” 方式，没有采用转发技术处理数据相关，并且同一个寄存器的读和写操作不能在同一个时钟周期内进行。请回答下列问题。

（1）若 int 型变量 x 的值为 −513，存放在寄存器 R1 中，则执行指令 `SHR R1` 后，R1 的内容是多少（用十六进制表示）?

（2）若某个时间段中有连续 4 条指令进入流水线，在其执行过程中没有发生任何阻塞，则执行这 4 条指令需要多少个时钟周期?

（3）若赋值语句为 `x=a+b`，x、a、b 均为 int 型变量，其存储单元地址分别为 [x]、[a]、[b]。对应指令序列及流水过程如下，I3 的 ID 段和 I4 的 IF 段被阻塞的原因各是什么?

```
I1  LOAD  R1, [a]
I2  LOAD  R2, [b]
I3  ADD   R1, R2
I4  STORE R2, [x]
```

最小反例·一核也能保存两线程状态

一个核心可保存两套 PC 和寄存器状态，按周期或等待事件交替执行，也可 SMT 跨线程发射。线程状态数与物理核心数不同，硬件多线程不要求先有多个核心。

指令	成功推进的阶段与周期	因依赖或前级冻结而等待
I1	IF1，ID2，EX3，M4，WB5	无
I2	IF2，ID3，EX4，M5，WB6	无
I3	IF3，ID7，EX8，M9，WB10	第 4～6 拍在 ID 端等待
I4	IF7，ID11，EX12，M13，WB14	第 4～6 拍未获准取指；第 8～10 拍在 ID 端等待

上表将原图时空格紧凑写成“阶段名 + 周期数”，另列等待状态：I3 第 4～6 拍在 ID 端等待，I4 第 4～6 拍尚未获准取指，第 8～10 拍又在 ID 端等待；这些等待状态保留了原图中的阻塞信息；其他未列时刻表示该指令尚未进入或已经离开流水线。

（4）若赋值语句为 `x=x*2+a`，x 和 a 均为 unsigned int 型变量，其存储单元地址分别为 [x]、[a]，执行这条语句至少需要多少个时钟周期？要求模仿上图，画出对应指令序列及流水线执行过程示意图。

解析。本题与例 5-8 的转发模型不同。全题统一采用如下约束：源寄存器若在第 w 拍 WB 写回，则消费者最早第 w+1 拍在 ID 完成取数并向后推进；在此之前可以停留于 ID 端等待，IF/ID 寄存器保持，后继取指也被阻塞。按题中过程，所有指令都要走完五段才算完成，即使某一段不产生实际数据读写。

（1）−513 的 16 位补码为 `FDFFH`，即 `1111 1101 1111 1111`。算术右移复制符号位，得到 `1111 1110 1111 1111`，即 `FEFFH`，真值 −257。表中“除以 2”说明算术右移的数量关系；对负奇数按位右移相当于向负无穷取整，不能套用 C 整数除法向零截断，答成 −256。

（2）无阻塞时，第一条用 5 拍，后 3 条每隔 1 拍完成，共 $5+4-1=8$ 拍。指令 Cache 与数据 Cache 分离，题干未给取指与数据访存争用，不应额外添加。

（3）I3 要读 I1 写入的 R1 和 I2 写入的 R2，存在两条 RAW 依赖。I1 在第 5 拍、I2 在第 6 拍 WB 写回，且同一寄存器不许同拍读写，所以 I3 在第 4～6 拍于 ID 端等待，第 7 拍才可完成取数，而非第 6 拍。I3 占用的 IF/ID 状态尚未释放时，若 I4 继续取指，会覆盖这条待推进指令，因此 I4 的 IF 也推迟到第 7 拍。后一处是前方数据等待向取指端传播，不能误说成 STORE 在 IF 就要读 R2。

等 I3 在第 10 拍 WB 写出和，I4 才可在第 11 拍 ID 读取 R2，随后经过 EX、M、WB，在第 14 拍流出。原图的两段空白应分别追溯到它等待的生产者与受阻的资源，不能把空白一概叫作访存延迟。

（4）用两个寄存器保存 x 和 a，让 x 先装入，以便尽早开始连续依赖的移位与相加：

```
I1  LOAD  R1, [x]
I2  LOAD  R2, [a]
I3  SHL   R1          // 也可用 ADD R1, R1
I4  ADD   R1, R2      // R2 <- 2*x + a
I5  STORE R2, [x]
```

课程连接·状态与控制分工

寄存器保留当前状态，控制器依状态与指令信息发出控制信号；保存状态与产生信号是不同功能。



Figure 5.4: 无转发且寄存器不能同拍读写：分别等待 WB 后的下一拍，最短 17 拍

I1 第 5 拍 WB，故 I3 最早第 6 拍 ID、第 9 拍 WB。I4 依赖 I3 的 R1，最早第 10 拍 ID、第 13 拍 WB；I2 已在第 6 拍写回，不会再额外拖延 I4。I5 最早第 14 拍 ID，至第 17 拍流出，故至少 17 拍。

这个排程还给出了下界证明：本题给定指令中，需要两次 LOAD、一次翻倍、一次相加和一次 STORE；x 的“LOAD→ 翻倍 → 相加 →STORE”链不能颠倒。即使其余工作都尽早完成，这条链也要求 ID 分别不早于第 6、10、14 拍；所列排程恰好达到下界。若先装 a 再装 x，关键链反而晚一拍。STORE 在第 16 拍 M 已写入存储器，但按原题五段完整流出的统一口径，答案仍为 17 拍。

5.12.3 流水线巩固

1. 原创补充：理想 5 段流水线周期 2 ns，完成 20 条指令，中间有 3 个导致整体延长的停顿周期，求总时间与实际吞吐率。
2. 原创补充：若一条指令的结果要到 MEM 末才形成，而消费者在下一拍 EX 开始使用，有 MEM→EX 转发，是否仍需等到生产者 WB 结束？若消费者 EX 与生产者 MEM 是同一拍呢？

答案。1. $(5+20-1+3)\times 2=54$ ns，吞吐率 $20/54\approx 0.370$ 条/ns。2. 前者不需要，可在下一拍使用 MEM 末的结果；后者来不及，需要推迟消费者。关键是结果形成时刻与使用时刻，而不是 “二者有相关” 这句判断本身。

5.13 CPU 内的状态与控制信息

CPU 既执行数据运算，又负责指令控制、操作控制、时间控制和中断处理。通用寄存器可由指令显式指定；PC、IR、MAR、MDR、微地址寄存器等承担特定组织功能，其中哪些对程序员可见要按 ISA 区分。PSW 保存条件码及控制状态，既包含算术结果信息，也可能包含中断允许、特权状态等。

取指周期根据 PC 取得指令并更新顺序地址；间址周期按需要取得

有效地址；执行周期完成指令规定动作；中断周期在满足条件时保存返回所需状态并进入服务程序。不是每条指令都经过全部这些周期。机器周期用于描述某种基本操作组织，时钟周期是时序基准，三者不能按名称认定固定倍数。

课程连接·单总线冲突

同一拍通常只能由一个源驱动总线，多个目的同时接收还需各自路径和写使能。

5.14 单总线数据通路的节拍安排

内部单总线通常同一时刻只允许一个部件驱动总线，多个寄存器可以在满足时序时同时接收同一总线值。ALU 若需要两个输入，而总线一拍只能给出一个源，须由暂存寄存器保存另一操作数；ALU 结果再暂存在 Z 一类寄存器中，后续送总线。

例如计算 $R_1 \leftarrow R_1 + R_2$，可依次组织：$R_1 \rightarrow Y$；$R_2 \rightarrow$ BUS 并使 ALU 执行 $Y + \text{BUS} \rightarrow Z$；$Z \rightarrow R_1$。如果把两个不同源同时接到同一总线，或在结果尚未稳定时提前写入目的寄存器，都违反数据通路条件。专用通路、多总线结构可以增加并行传送能力，但还要检查每条路径和写使能的实际连接。

5.14.1 取指与间址微操作的依赖

取指中常见 PC $\rightarrow$ MAR、$M[\text{MAR}] \rightarrow$ MDR、MDR $\rightarrow$ IR 以及 PC 顺序更新。访存完成前，MDR 中的旧内容不能当作本条新指令。指令进入 IR 后，操作码才可用于译码；若地址字段本身用于间接寻址，还需把它送 MAR，再读出有效地址。

能否把 PC 加 1 与读存储器放在同一节拍，取决于 PC 增量器是否独立、总线是否被占用以及控制器的时序方案。课件节拍表是某一具体组织的实例，应先理解依赖关系，再照图列允许的并行动作。

5.15 硬布线控制器从何处获得控制信号

硬布线控制器把指令操作码、时序节拍、机器周期状态和有关条件标志输入组合逻辑，生成各寄存器写使能、总线输出使能、ALU 操作选择及存储器读写控制。设计时先分析指令微操作，再安排节拍，最后把“某指令、某周期、某节拍、某条件下发出某信号”合成为逻辑表达式。

例如某控制信号在指令 A 的 T_1 或指令 B 的 T_2 发出，可概括为 $C = A \cdot T_1 + B \cdot T_2$；若还取决于 ZF，应把条件乘入对应项。这里的加、乘是逻辑或、与。硬布线速度通常较快，但指令和控制条件增加时，设计、修改及验证复杂度也增加。

5.16 微程序控制：区分四种地址

主存地址定位机器指令或数据，PC 保存程序级指令地址；控制存储器 CM 保存微指令，CMAR 保存微指令地址；CMDR 保存当前微指令，操作控制字段输出微命令，顺序控制字段参与确定下一微地址。主存和控存是不同逻辑空间，机器指令地址与微指令地址也不是同一数值体系。

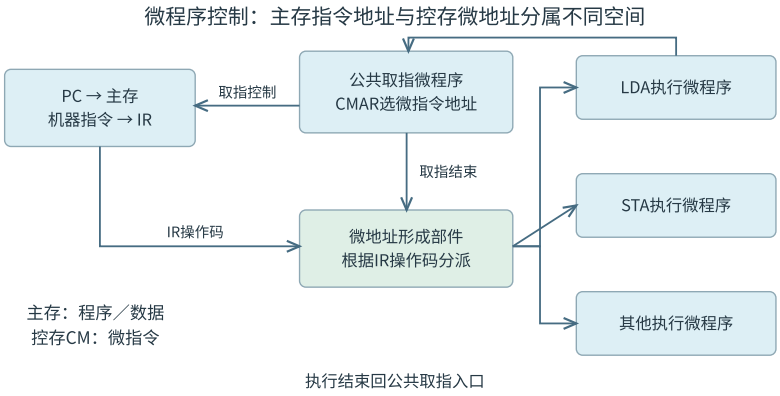


Figure 5.5: 公共取指微程序先运行，再由操作码映射到相应执行微程序入口

取指微程序可以由全部机器指令共享。取指结束后，根据 IR 操作码形成执行微程序入口；执行微程序结束后，通常回到公共取指入口。遇到条件转移、间址或中断时，还可能根据测试条件和周期状态选择相应微程序分支。

5.16.1 微指令编码与下地址形成

直接控制法为每个控制信号分配一位，译码简单、并行性强而字长较长。字段直接编码把互斥微命令分到同一字段，不同字段可并行；一个含 k 个可选微命令的字段若需额外表示“本字段不发命令”，至少 $\lceil \log_2(k+1) \rceil$ 位。相关编码还可利用字段之间的依赖进一步压缩，但增加译码层次。

水平型微指令一般允许同拍多个微操作并行，控制字段较宽；垂直型采用更强编码，字长较短而执行微操作的并行程度受到限制。这里的水平、垂直是控制信息组织方式，不是微程序在纸上的排版方向。

下一微地址可由当前微指令下地址字段直接给出，也可由微地址计数器顺序增量、指令操作码映射、条件测试或硬件强制入口形成。下地址位宽按控存实际需要区分的微地址数量估算，而不是按机器指令条数直接取对数。

设 32 条机器指令，各自执行段平均 4 条微指令，共享取指段 2 条，且这些段互不重叠，则至少需存储 $32 \times 4 + 2 = 130$ 条微指令，顺序地址字段至少 8 位。若微程序共享其他公共段、还含中断段或间址段，须据实际布局重新统计。

5.16.2 微程序控制单元的设计步骤

先列每条机器指令所需微操作并安排可并行的节拍，再补入“读下一微指令、形成微地址”所需的控制动作；选择控制字段编码和下地址形成方式，确定微指令字长，最后分配控存地址并编写各条微指令码。用当前微指令控制数据通路的同时，还应确保下一个微地址及时稳定。

静态微程序通常固化于 ROM，动态微程序允许通过可改写控存调整解释规则。毫微程序在微指令之下再设一层解释，以进一步压缩或组织控制信息。微程序修改依赖硬件支持及控制存储器类型，不能把修改普通程序文件等同于修改 CPU 微程序。

不变量·微指令字段各有职责

控制字段决定本拍发出的微命令，顺序字段参与形成下一微地址；操作码映射只用于需要选择微程序入口的阶段。

5.17 五段流水线中的装入、存储和转移

课程连接·值何时可用

装入结果在访存完成后可用；旁路路径和使用时刻共同决定需要几个气泡。

IF 取指，ID 译码并读取源寄存器，EX 进行算术运算或计算有效地址，MEM 访问数据存储器，WB 写回寄存器。不是每条指令都在每一段执行同样的实质工作，但简化固定五段模型可保留阶段位置以统一时序。

装入指令在 EX 算地址、MEM 取数据、WB 写寄存器；存储指令在 EX 算地址、MEM 写数据，通常没有寄存器写回；算术指令在 EX 产生结果，后续沿阶段寄存器传递；条件转移在哪段确定方向和目标，应按题设明确。阶段寄存器除数据外还须保存后续需要的目的寄存器号、控制位和异常信息。

在理想等长 k 段、每段时间 Δt、连续输入 n 条指令的模型中，总时间 $(k+n-1)\Delta t$，吞吐率 $n/[(k+n-1)\Delta t]$，相对完全串行的加速比 $nk/(k+n-1)$，效率 $n/(k+n-1)$。这些公式的前提是无冒险、无额外等待且阶段时间相同；不等长阶段通常按最慢段及寄存器开销确定时钟。

5.17.1 硬件多线程与共享内存组织

粗粒度多线程通常在较长停顿时切换线程，切换后还可能需要清空并重新填充流水线；细粒度多线程按较短固定粒度交替发射不同线程指令，用更多独立指令掩盖停顿；同时多线程 SMT 允许同一周期从多个线程选择指令占用不同发射槽。每个线程需要自己的体系结构状态，而功能部件等资源可以共享。

共享内存多处理器共享物理地址空间，可用 LOAD/STORE 访问共同主存；多计算机系统各有私有主存，通过消息传递通信。多核表示一块芯片集成多个处理核，是物理集成方式；SIMD 表示一个指令流控制多个数据通路，是并行控制方式，不能把二者当作同一分类轴。

5.17.2 三种硬件多线程的发射比较

时钟	细粒度：每拍选择一个线程	粗粒度：长停顿时换线程	SMT：一拍可选多个线程
i	A 的 j、j+1	A 的 j、j+1	A 的 j、j+1；B 的 k、k+1
i+1	B 的 k、k+1	A 的 j+2、j+3，发现 Cache 缺失	A 的 j+2；B 的 k+2；C 的 m
i+2	A 的 j+2、j+3	调度，从 A 切换到 B	A 的 j+3；C 的 m+1、m+2
i+3	B 的 k+2、k+3	B 的 k、k+1	按就绪指令及资源继续选择

该示例中的细粒度多线程允许同一拍从选中的线程发射多条指令；关键是这一拍仅选择一个线程。SMT 则可以在同一拍把不同线程的就绪指令安排到多个执行槽。粗粒度切换时可能需要重填流水线，因此切换的成本与粒度相关。

5.17.3 计算与辨析自查

1. PC、MAR、CMAR 保存的地址分别属于什么空间？2. 含 7 个互斥微命令及空操作的字段需要几位？3. 理想 5 段流水线执行 20 条指令需几拍？4. 为什么 load-use 冒险可能在有转发时仍需停顿？

答案。1. PC 为机器指令地址，MAR 为本次主存访问地址，CMAR 为控存微指令地址。2. 3 位，正好 8 种编码。3. $5+20-1=24$ 拍。4.

使用者需要操作数时，前一装入指令的数据可能尚未从 MEM 产生，转发只能传已有数据。

不变量·调度下界来自不可颠倒链

无转发且不许同拍读写时，LOAD→翻倍 → 相加 →STORE 链的各次 ID 必须晚于相应生产者 WB。先给关键链下界，再排出达到该下界的合法时序，才能证明 “最少几拍”。

5.18 主动回忆自测

以下均为自拟检查。沿章导图，先说明对象与前提，再写结论。

1. PC、IR、MAR、MDR 分别暂存什么？为何 MAR 与 MDR 位宽无须相同？
2. 一条指令是否总经过取指、间址、执行与中断四阶段？
3. 单内部总线同拍可否由两个不同源送两个不同目的？同一源送两个目的呢？
4. 机器指令、微指令与微命令各控制或表示哪个层次？PC 与 CMAR 有何区别？
5. 某互斥字段有 4 个微命令，为什么按通常约定至少需 3 位？
6. 故障处理后重执行当前指令，与外部中断后返回后继位置，依据是什么？
7. 理想 5 段、每段 2ns、20 条指令加 3 拍整体停顿，总时间和吞吐率分别是多少？
8. 在 EX 形成的 ALU 值与 MEM 末形成的 load 值，紧邻消费者所需等待为何可能不同？
9. 寄存器重命名能否消除真实 RAW？单核能否采用硬件多线程？
10. SIMD、MIMD、超标量与 SMT 分别按哪种并行维度分类？共享地址空间是否使读改写自动原子？

5.18.1 自测答案

1. PC 与后继取指位置有关，IR 存当前指令，MAR 存访存地址，MDR 存与主存交换的值；地址位数取决于可寻址单元数，MDR 位数取决于交换数据通路。2. 不总是；间接寻址、外部中断等有相应条件。3. 不同源通常冲突；同源多目的在独立写使能和实际路径允许时可以。4. 机器指令规定 ISA 操作，微指令组织一个控制步，微命令驱动具体部件；PC 访问机器程序，CMAR 访问控制存储器。5. 4 个命令加 “不发本组命令” 共 5 种状态，需 3 位。6. 故障尚未完成原定操作，可恢复时需要重试；外部中断通常发生在指令完成边界，但具体保存 PC 规则依 ISA。7. $T=(5+20-1+3)\times 2=54$ ns，吞吐率约 0.370 条/ns。8. 按结果形成时刻、消费者使用时刻及转发路径判断；数据尚未形成就无法转发。9. 不能消除真实值依赖；单核可保存多套线程状态并共享执行资源。10. SIMD/MIMD 按指令流与数据流，超标量按每周期多发射，SMT 按同周期跨线程利用执行部件；共享地址空间仍需原子操作与同步。